arXiv:2509.25164v5 [cs.CV] 16 Mar 2026

YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection

Ranjan Sapkota[1] **Rahul Harsha Cheppally**[2] **Ajay Sharda**[2] **Manoj Karkee**[1]
[1]Cornell University, Biological & Environmental Engineering, Ithaca, NY 14850, USA
rs2672@cornell.edu, mk2684@cornell.edu
[2]Kansas State University, Department of Biological and Agricultural Engineering, Manhattan, KS 66502, USA

March 17, 2026

Abstract

This study presents a comprehensive analysis of Ultralytics YOLO26, highlighting its key architectural enhancements and performance benchmarking for real-time edge object detection. YOLO26, released in September 2025, stands as the newest and most advanced member of the YOLO family, purpose-built to deliver efficiency, accuracy, and deployment readiness on edge and low-power devices. The paper sequentially details YOLO26's architectural innovations, including the removal of Distribution Focal Loss (DFL), adoption of end-to-end NMS-free inference, integration of ProgLoss and Small-Target-Aware Label Assignment (STAL), and the introduction of the MuSGD optimizer for stable convergence. Beyond architecture, the study positions YOLO26 as a multi-task framework, supporting object detection, instance segmentation, pose/keypoints estimation, oriented detection, and classification. We present performance benchmarks of YOLO26 on edge devices such as NVIDIA Jetson Nano and Orin, comparing its results with YOLOv8, YOLOv11, YOLOv12, YOLOv13, and transformer-based detectors. This paper further explores real-time deployment pathways, flexible export options (ONNX, TensorRT, CoreML, TFLite), and quantization for INT8/FP16. Practical use cases of YOLO26 across robotics, manufacturing, and IoT are highlighted to demonstrate cross-industry adaptability. Finally, insights on deployment efficiency and broader implications are discussed, with future directions for YOLO26 and the YOLO lineage outlined.

Keywords YOLO26 · Edge AI · Multi-task Object Detection · NMS-free Inference · Small Target Recognition · You Only Look Once · Object Detection · MuSGD Optimizer

1 Introduction

Object detection has emerged as one of the most critical tasks in computer vision, enabling machines to localize and classify multiple objects within an image or video stream [1, 2]. From autonomous driving and robotics to surveillance, medical imaging, agriculture, and smart manufacturing, real-time object detection algorithms serve as the backbone of artificial intelligence (AI) applications [3, 4]. Among these algorithms, the You Only Look Once (YOLO) family has established itself as the most influential series of models for real-time object detection, combining accuracy with unprecedented inference speed [5, 6, 7, 7]. Since its introduction in 2016, YOLO has evolved through numerous architectural revisions, each addressing limitations of its predecessors while integrating cutting-edge advances in neural network design, loss functions, and deployment efficiency [5].

The release of YOLO26 in September 2025 marks the newest milestone in the YOLO lineage, shifting the design emphasis from incremental architectural complexity toward deployment-oriented simplification—most notably through streamlined regression, end-to-end prediction behavior, and training-time refinements enabled by novel optimization. This edge-first philosophy is reflected in the comparative accuracy–latency trends shown in Fig. 1a, where Ultralytics reports YOLO26's COCO mAP(50-95) versus latency performance (T4, TensorRT10, FP16) against a broad set of prior

YOLO variants (YOLO11, YOLOv10, YOLOv9, YOLOv8, YOLOv7, YOLOv6-3.0, YOLOv5) as well as competitive real-time detectors (PP-YOLOE+, DAMO-YOLO, and RTMDet). Complementing this, Fig. 1b positions YOLO26 on the same COCO mAP(50-95) versus end-to-end latency axis against transformer-style real-time baselines (YOLOv10 and the RT-DETR family), underscoring that YOLO26 aims to retain high detection quality while reducing overall pipeline delay, an especially relevant trade-off for low-power and latency-sensitive edge devices.

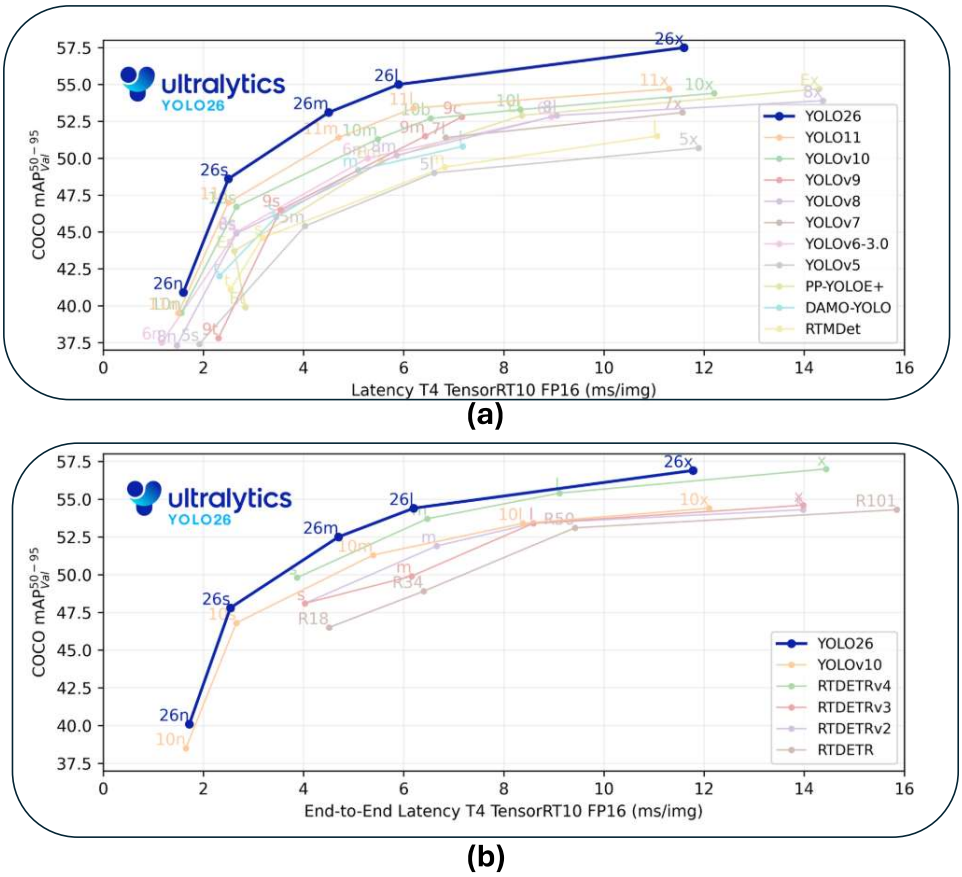


Figure 1: Performance comparison of YOLO26 under TensorRT FP16 on an NVIDIA T4 GPU (Source Link). (a) COCO mAP(50–95) versus inference latency (ms/image), comparing YOLO26 with earlier YOLO versions and other real-time detectors, highlighting its improved accuracy–speed trade-off. (b) COCO mAP(50–95) versus end-to-end latency, comparing YOLO26 with YOLOv10 and RT-DETR variants, illustrating its advantage in overall pipeline efficiency.

Table 1 provides a detailed comparison of YOLO models from version YOLOv1 to YOLOv13 and YOLO26, highlighting their release years, key architectural innovations, performance enhancements, and development frameworks.

The YOLO framework was first proposed by Joseph Redmon and colleagues in 2016, introducing a paradigm shift in object detection [8]. Unlike traditional two-stage detectors such as R-CNN [18] and Faster R-CNN [19], which separated region proposal from classification, YOLO formulated detection as a single regression problem [20]. By directly predicting bounding boxes and class probabilities in one forward pass through a convolutional neural network (CNN), YOLO achieved real-time speeds while maintaining competitive accuracy [21, 20]. This efficiency made YOLOv1 highly attractive for applications where latency was a critical factor, including robotics, autonomous navigation, and live video analytics. Subsequent versions YOLOv2 (2017) [9]and YOLOv3 (2018) [10] significantly improved accuracy while retaining real-time performance. YOLOv2 introduced batch normalization, anchor boxes, and multi-scale training, which increased robustness across varying object sizes. YOLOv3 leveraged a deeper architecture based on Darknet-53, along with multi-scale feature maps for better small-object detection. These enhancements made YOLOv3 the de facto standard for academic and industrial applications for several years [22, 5, 23].

As the demand for higher accuracy grew, especially in challenging domains such as aerial imagery, agriculture, and medical analysis, YOLO models diversified into more advanced architectures. YOLOv4 (2020) [11] introduced

Table 1: Summary of YOLOv1 to YOLOv13 and YOLOv26 models: release year, architecture, innovations, frameworks

Model (Year)	Key Architectural Innovation and Contribution	Tasks	Framework
YOLOv1 (2015) [8]	First unified single-stage object detector (one network for bounding boxes + class probabilities).	Object Detection, Classification	Darknet
YOLOv2 (2016) [9]	Multi-scale training introduced; anchor box dimension clustering for improved prior boxes (YOLO9000 joint detection/classification).	Object Detection, Classification	Darknet
YOLOv3 (2018) [10]	Deeper Darknet-53 backbone with residual connections; added SPP module and multi-scale feature fusion for small object detection.	Object Detection, Multi-scale Detection	Darknet
YOLOv4 (2020) [11]	Mish activation function adopted; CSPDarknet-53 backbone (Cross-Stage Partial networks) for enhanced feature reuse.	Object Detection, Object Tracking	Darknet
YOLOv5 (2020) (Source Link)	PyTorch implementation by Ultralytics; anchor-free detection head option; used SiLU (Swish) activation and PANet neck for feature aggregation.	Object Detection, Instance Segmentation (limited)	PyTorch (Ultralytics)
YOLOv6 (2022) [12]	EfficientRep backbone with embedded self-attention; introduced anchor-free object detection mode for efficiency.	Object Detection, Instance Segmentation	PyTorch
YOLOv7 (2022) [13]	Extended ELAN (E-ELAN) backbone with model re-parameterization; integrated transformer-based modules for broader tasks (e.g. tracking).	Object Detection, Object Tracking, Instance Segmentation	PyTorch
YOLOv8 (2023) (Source Link)	Ultralytics next-gen model; new C2f backbone and decoupled head; incorporated generative techniques (GAN-based augmentation) and fully anchor-free design.	Object Detection, Instance Segmentation, Panoptic Segmentation, Keypoint Estimation	PyTorch (Ultralytics)
YOLOv9 (2024) [14]	Introduced Programmable Gradient Information (PGI) for selective learning; proposed G-ELAN (an enhanced ELAN architecture) for improved feature extraction.	Object Detection, Instance Segmentation	PyTorch
YOLOv10 (2024) [15]	Achieved end-to-end NMS-free detection via a consistent dual-assignment training strategy (removing post-processing).	Object Detection	PyTorch
YOLO11 (2024) (Source Link)	Added C3k2 CSP bottlenecks (smaller kernel CSP blocks) throughout backbone/neck for efficiency; retained SPPF and introduced C2PSA (CSP with Spatial Attention) module to focus on important regions. Extended YOLO to pose estimation and oriented object detection tasks.	Object Detection, Instance Segmentation, Pose Estimation, Oriented Detection	PyTorch (Ultralytics)
YOLOv12 (2025) [16]	Attention-centric architecture: introduced an efficient area attention module (global self-attention with low complexity) and Residual ELAN (R-ELAN) blocks to improve feature aggregation, achieving transformer-level accuracy at YOLO speeds.	Object Detection	PyTorch
YOLOv13 (2025) [17]	Hypergraph-based Adaptive Correlation Enhancement (HyperACE) module to capture global high-order feature interactions; Full-Pipeline Aggregation-Distribution (FullPAD) scheme for enhanced feature flow throughout the network; utilized depthwise-separable convolutions to reduce complexity.	Object Detection	PyTorch
YOLOv26 (2025) (Source Link)	Ultralytics edge-optimized model: eliminated NMS with a native end-to-end predictor; removed DFL (Distribution Focal Loss) for simpler, faster inference; introduced MuSGD optimizer (SGD+Muon hybrid) for stable and quick convergence; significantly improved small-object accuracy and up to 43% faster CPU inference for deployment on low-power devices.	Object Detection, Instance Segmentation, Pose Estimation, Oriented Detection, Classification	PyTorch (Ultralytics)

Cross-Stage Partial Networks (CSPNet), improved activation functions like Mish, and advanced training strategies including mosaic data augmentation and CIoU loss. YOLOv5 (Ultralytics, 2020), though unofficial, gained immense popularity due to its PyTorch implementation, extensive community support, and simplified deployment across diverse platforms. YOLOv5 also brought modularity, making it easier to adapt for segmentation, classification, and edge applications. Further developments included YOLOv6[12] and YOLOv7 [13] (2022), which integrated advanced optimization techniques, parameter-efficient modules, and transformer-inspired blocks. These iterations pushed YOLO closer to state-of-the-art (SoTA) accuracy benchmarks while retaining a focus on real-time inference. The YOLO ecosystem, by this point, had firmly established itself as the leading family of models in object detection research and deployment.

Ultralytics, the primary maintainer of modern YOLO releases, redefined the framework with YOLOv8 (2023) [24]. YOLOv8 featured a decoupled detection head, anchor-free predictions, and refined training strategies, resulting in substantial improvements in both accuracy and deployment versatility [25]. It was widely adopted in industry due to its clean Python API, compatibility with TensorRT, CoreML, and ONNX, and availability of variants optimized for speed versus accuracy trade-offs (nano, small, medium, large, and extra-large). YOLOv9 [14], YOLOv10 [15], and YOLO11 followed in rapid succession, each iteration pushing the boundaries of architecture and performance. YOLOv9 introduced GELAN (Generalized Efficient Layer Aggregation Network) and Progressive Distillation, combining efficiency with higher representational capacity. YOLOv10 focused on balancing accuracy and inference latency with hybrid task-aligned assignments. YOLOv11 further refined Ultralytics' vision, offering higher efficiency on GPUs while maintaining strong small-object performance [5]. Together, these models cemented Ultralytics' reputation for producing production-ready YOLO releases tailored to modern deployment pipelines.

Following YOLO11, alternative versions YOLOv12[16] and YOLOv13 [17] introduced attention-centric designs and advanced architectural components that sought to maximize accuracy across diverse datasets. These models explored multi-head self-attention, improved multi-scale fusion, and stronger training regularization strategies. While they offered strong benchmarks, they retained reliance on Non-Maximum Suppression (NMS) and Distribution Focal Loss (DFL), which introduced latency overhead and export challenges, especially for low-power devices. The limitations of NMS-based post-processing and complex loss formulations motivated the development of YOLO26 (Ultralytics YOLO26 Official Source). By September 2025, at the YOLO Vision 2025 event in London, Ultralytics unveiled YOLO26 as a next-generation model optimized for edge computing, robotics, and mobile AI.

YOLO26 is engineered around three guiding principles simplicity, efficiency, and innovation and the overview in Figure 2 situates these choices alongside its five supported tasks: object detection, instance segmentation, pose/keypoints detection, oriented detection, and classification. On the inference path, YOLO26 eliminates NMS, producing native end-to-end predictions that remove a major post-processing bottleneck, reduce latency variance, and simplify threshold tuning across deployments. On the regression side, it removes DFL, turning distributional box decoding into a lighter, hardware-friendly formulation that exports cleanly to ONNX, TensorRT, CoreML, and TFLite a practical win for edge and mobile pipelines. Together, these changes yield a leaner graph, faster cold-start, and fewer runtime dependencies, which is particularly beneficial for CPU-bound and embedded scenarios. Training stability and small-object fidelity are addressed through ProgLoss (progressive loss balancing) and STAL (small-target-aware label assignment). ProgLoss adaptively reweights objectives to prevent domination by easy examples late in training, while STAL prioritizes assignment for tiny or occluded instances, improving recall under clutter, foliage, or motion blur conditions common in aerial, robotics, and smart-camera feeds. Optimization is driven by MuSGD, a hybrid that blends the generalization of SGD with momentum/curvature behaviors inspired by Muon-style methods, enabling faster, smoother convergence and more reliable plateaus across scales.

Functionally, as highlighted again in Figure 2, YOLO26's five capabilities share a unified backbone/neck and streamlined heads:

- **Object Detection:** Anchor-free, NMS-free boxes and scores
- **Instance Segmentation:** Lightweight mask branches coupled to shared features;
- **Pose/Keypoints Detection:** Compact keypoint heads for human or part landmarks
- **Oriented Detection:** Rotated boxes for oblique objects and elongated targets
- **Classification:** Single-label logits for pure recognition tasks.

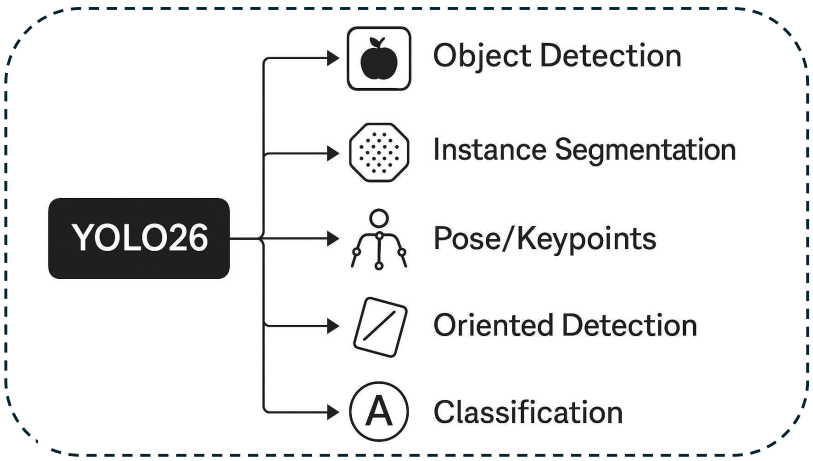


Figure 2: YOLO26 unified architecture supports five key vision tasks object detection, instance segmentation, pose/keypoints detection, oriented detection, and classification.

This consolidated design allows multi-task training or task-specific fine-tuning without architectural rework, while the simplified exports preserve portability across accelerators. In sum, YOLO26 advances the YOLO lineage by pairing end-to-end inference and DFL-free regression with ProgLoss, STAL, and MuSGD, yielding a model that is faster to deploy, steadier to train, and broader in capability as visually summarized in Figure 2.

2 Architectural Enhancements in YOLO26

The architecture of YOLO26 follows a streamlined and efficient pipeline that has been purpose-built for real-time object detection across edge and server platforms. As illustrated in Figure 3, the process begins with the ingestion of input data in the form of images or video streams, which are first passed through preprocessing operations including resizing and normalization to standard dimensions suitable for model inference. The data is then fed into the backbone feature extraction stage, where a compact yet powerful convolutional network captures hierarchical representations of visual patterns. To enhance robustness across scales, the architecture generates multi-scale feature maps (Figure 3) that preserve semantic richness for both large and small objects. These feature maps are then merged within a lightweight feature fusion neck, where information is integrated in a computationally efficient manner. Detection-specific processing occurs in the direct regression head, which, unlike prior YOLO versions, outputs bounding boxes and class probabilities without relying on Non-Maximum Suppression (NMS). This end-to-end NMS-free inference (Figure 3) eliminates post-processing overhead and accelerates deployment. Training stability and accuracy are reinforced by ProgLoss balancing and STAL assignment modules, which ensure equitable weighting of loss terms and improved detection of small targets. Model optimization is guided by the MuSGD optimizer, combining the strengths of SGD and Muon for faster and more reliable convergence. Deployment efficiency is further enhanced through quantization, with support for FP16 and INT8 precision, enabling acceleration on CPUs, NPUs, and GPUs with minimal accuracy degradation. Finally, the pipeline culminates in the generation of output predictions, including bounding boxes and class assignments that can be visualized overlaid on the input image. Overall, the architecture of YOLO26 demonstrates a carefully balanced design philosophy that simultaneously advances accuracy, stability, and deployment simplicity.

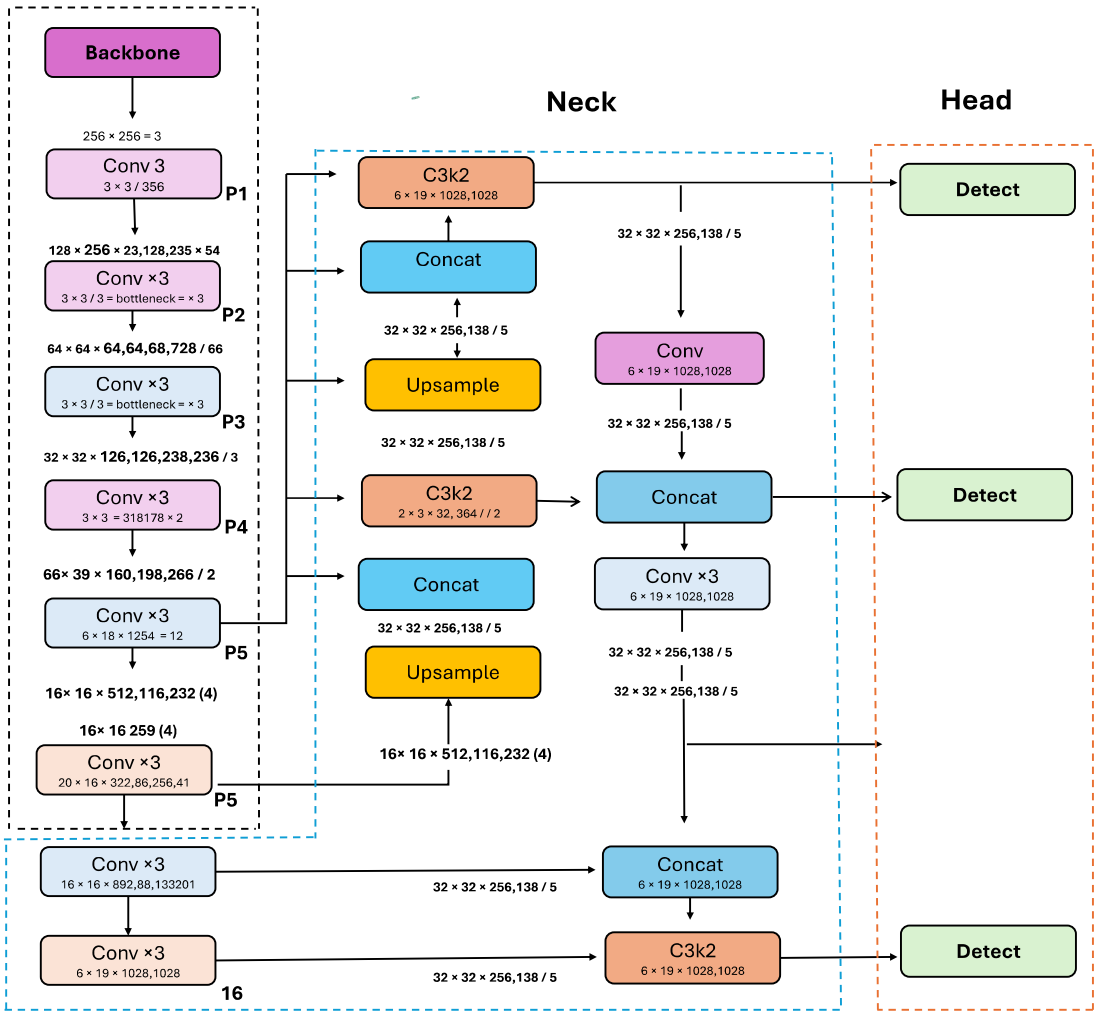


Figure 3: Core architecture diagram of Ultralytics YOLO26 object detection and segmentation algorithm

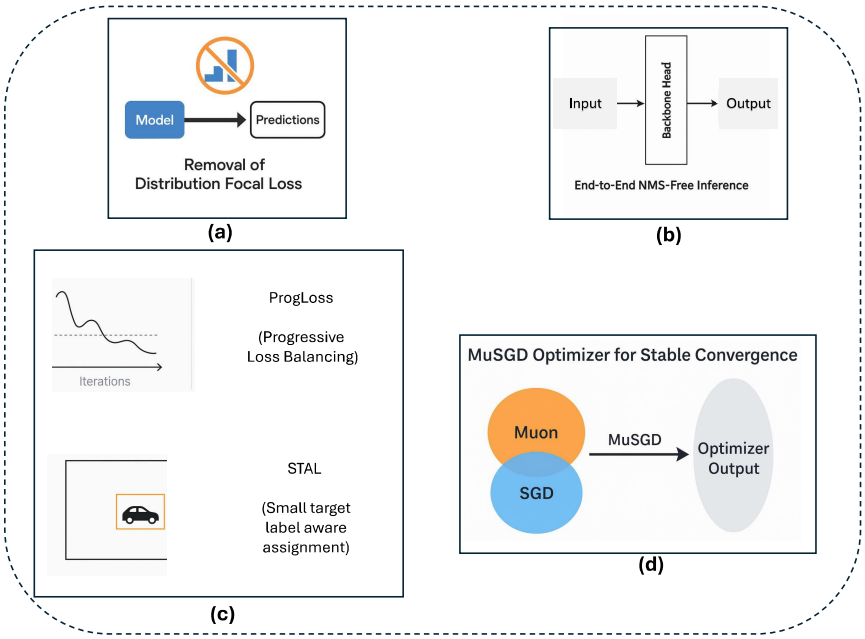


Figure 4: Key architectural enhancements in YOLO26: (a) Removal of Distribution Focal Loss (DFL) streamlines bounding box regression, boosting efficiency and export compatibility. (b) End-to-end NMS-free inference eliminates post-processing bottlenecks, enabling faster and simpler deployment. (c) ProgLoss and STAL enhance training stability and significantly improve small-object detection accuracy. (d) The MuSGD optimizer combines SGD and Muon strengths, achieving faster, more stable convergence in training.

YOLO26 introduces several key architectural innovations that differentiate it from prior generations of YOLO models. These enhancements not only improve training stability and inference efficiency but also fundamentally reshape the deployment pipeline for real-time edge devices. In this section, we describe four major contributions of YOLO26: (i) the removal of Distribution Focal Loss (DFL), (ii) the introduction of end-to-end Non-Maximum Suppression (NMS)-free inference, (iii) novel loss function strategies including Progressive Loss Balancing (ProgLoss) and Small-Target-Aware Label Assignment (STAL), and (iv) the development of the MuSGD optimizer for stable and efficient convergence. Each of these architectural enhancements is discussed in detail, with comparative insights highlighting their advantages over earlier YOLO versions such as YOLOv8, YOLOv11, YOLOv12, and YOLOv13.

2.1 Removal of Distribution Focal Loss (DFL)

One of the most significant architectural simplifications in YOLO26 is the removal of the Distribution Focal Loss (DFL) module (Figure 4a), which had been present in prior YOLO releases such as YOLOv8 and YOLOv11. DFL was originally designed to improve bounding box regression by predicting probability distributions for box coordinates, thereby allowing more precise localization of objects. While this strategy demonstrated accuracy gains in earlier models, it also introduced non-trivial computational overhead and export difficulties. In practice, DFL required specialized handling during inference and model export, which complicated deployment pipelines targeting hardware accelerators such as ONNX, CoreML, TensorRT, or TFLite.

By eliminating DFL, YOLO26 simplifies the model's architecture, making bounding box prediction a more straightforward regression task without sacrificing performance. Comparative analysis indicates that YOLO26 achieves comparable or superior accuracy to DFL-based YOLO models, particularly when combined with other innovations such as ProgLoss and STAL. Moreover, the removal of DFL significantly reduces inference latency and improves cross-platform compatibility. This makes YOLO26 more suitable for edge AI scenarios, where lightweight and hardware-friendly models are paramount.

In contrast, models such as YOLOv12 and YOLOv13 retained DFL in their architectures, which limited their applicability on constrained devices despite strong accuracy benchmarks on GPU-rich environments. YOLO26 therefore marks a decisive step toward aligning state-of-the-art object detection performance with the realities of mobile, embedded, and industrial applications.

2.2 End-to-End NMS-Free Inference

Another groundbreaking feature of YOLO26 is its native support for end-to-end inference without Non-Maximum Suppression (NMS) (Refer to Figure 4b). Traditional YOLO models, including YOLOv8 through YOLOv13, rely heavily on NMS as a post-processing step to filter out duplicate predictions by retaining only the bounding boxes with the highest confidence scores. While effective, NMS adds additional latency to the pipeline and requires manually tuned hyperparameters such as the Intersection-over-Union (IoU) threshold. This dependence on a handcrafted post-processing step introduces fragility in deployment pipelines, especially for edge devices and latency-sensitive applications.

YOLO26 fundamentally redesigns the prediction head to produce direct, non-redundant bounding box predictions without the need for NMS. This end-to-end design not only reduces inference complexity but also eliminates the dependency on hand-tuned thresholds, thereby simplifying integration into production systems. Comparative benchmarks demonstrate that YOLO26 achieves faster inference speeds than YOLOv11 and YOLOv12, with CPU inference times reduced by up to 43% for the nano model. This makes YOLO26 particularly advantageous for mobile devices, UAVs, and embedded robotics platforms where milliseconds of latency can have substantial operational impacts.

Beyond speed, the NMS-free approach improves reproducibility and deployment portability, as models no longer require extensive post-processing code. While other advanced detectors such as RT-DETR and Sparse R-CNN have experimented with NMS-free inference, YOLO26 represents the first YOLO release to adopt this paradigm while maintaining YOLO's hallmark balance between speed and accuracy. Compared to YOLOv13, which still depends on NMS, YOLO26's end-to-end pipeline stands out as a forward-looking architecture for real-time detection.

2.3 ProgLoss and STAL: Enhanced Training Stability and Small-Object Detection

Training stability and small-object recognition remain persistent challenges in object detection. YOLO26 addresses these through the integration of two novel strategies: Progressive Loss Balancing (ProgLoss) and Small-Target-Aware Label Assignment (STAL), as depicted in Figure (Figure 4c)

ProgLoss dynamically adjusts the weighting of different loss components during training, ensuring that the model does not overfit to dominant object categories while underperforming on rare or small classes. This progressive rebalancing improves generalization and prevents instability during later epochs of training. STAL, on the other hand, explicitly prioritizes label assignments for small objects, which are particularly difficult to detect due to their limited pixel representation and susceptibility to occlusion. Together, ProgLoss and STAL provide YOLO26 with a substantial accuracy boost on datasets with small or occluded objects, such as COCO and UAV imagery benchmarks.

Comparatively, earlier models such as YOLOv8 and YOLOv11 did not incorporate such targeted mechanisms, often requiring dataset-specific augmentations or external training tricks to achieve acceptable small-object performance. YOLOv12 and YOLOv13 attempted to address this gap through attention-based modules and enhanced multi-scale feature fusion; however, these solutions increased architectural complexity and inference costs. YOLO26 achieves similar or superior improvements with a more lightweight approach, reinforcing its suitability for edge AI applications. By integrating ProgLoss and STAL, YOLO26 establishes itself as a robust small-object detector while maintaining the efficiency and portability of the YOLO family.

2.4 MuSGD Optimizer for Stable Convergence

A final innovation in YOLO26 is the introduction of the MuSGD optimizer (Figure 4d), which combines the strengths of Stochastic Gradient Descent (SGD) with the recently proposed Muon optimizer, a technique inspired by optimization strategies used in large language model (LLM) training. MuSGD leverages the robustness and generalization capacity of SGD while incorporating adaptive properties from Muon, enabling faster convergence and more stable optimization across diverse datasets.

This hybrid optimizer reflects an important trend in modern deep learning: the cross-pollination of advances between natural language processing (NLP) and computer vision. By borrowing from LLM training practices (e.g., Kimi K2 by Moonshot AI), YOLO26 benefits from stability enhancements that were previously unexplored in the YOLO lineage. Empirical results show that MuSGD enables YOLO26 to reach competitive accuracy with fewer training epochs, reducing both training time and computational cost.

Previous YOLO versions, including YOLOv8 through YOLOv13, relied on standard SGD or AdamW variants. While effective, these optimizers required extensive hyperparameter tuning and sometimes exhibited unstable convergence, particularly on datasets with high variability. In comparison, MuSGD improves reliability while preserving YOLO's lightweight training ethos. For practitioners, this translates into shorter development cycles, fewer training restarts, and more predictable performance across deployment scenarios. By integrating MuSGD, YOLO26 positions itself as not

only an inference-optimized model but also a training-friendly architecture for researchers and industry practitioners alike.

3 Benchmarking and Comparative Analysis

3.1 Detection and Segmentation Performance Metrics

As summarized in the Detection results in Table 2, YOLO26 consistently improves COCO mAP(50–95) as model scale increases, while maintaining predictable and low inference latency across CPU (ONNX) and GPU (TensorRT) runtimes. In particular, YOLO26-m and YOLO26-l achieve strong detection accuracy above 53% and 55% mAP, respectively, at substantially lower latency than transformer-based alternatives, reflecting the benefits of its NMS-free inference path and simplified regression design. The same table further highlights favorable scaling behavior in parameters and FLOPs, reinforcing YOLO26's efficiency across deployment targets.

Beyond detection, the Segmentation results in Table 2 demonstrate that YOLO26 retains these advantages in multi-task settings. Across nano to extra-large variants, YOLO26-seg models deliver competitive box and mask mAP while preserving manageable computational cost and real-time throughput, even under end-to-end evaluation. When contrasted with architectures such as YOLOv10 and RT-DETR variants, which rely on heavier transformer encoders, YOLO26 exhibits a more balanced accuracy–latency profile, particularly for edge and CPU-bound inference. Taken together, the detection and segmentation benchmarks in Table 2 show that YOLO26 is not merely an incremental refinement, but a deployment-oriented evolution of the YOLO family, effectively bridging efficiency-focused design and high-accuracy real-time perception under stringent latency constraints.

Table 2: Ultralytics YOLO26 performance metrics (640 px). Detection (top) and instance segmentation (bottom) results report COCO validation accuracy, end-to-end (e2e) scores where applicable, and speed on CPU (ONNX) and NVIDIA T4 (TensorRT10 FP16), along with model size (params) and compute (FLOPs).

Detection							
Model	**Size (px)**	**mAP$_{val}$ 50–95**	**mAP$_{val}$ 50–95 (e2e)**	**Speed CPU ONNX (ms)**	**Speed T4 TRT10 (ms)**	**Params (M)**	**FLOPs (B)**
YOLO26n	640	40.9	40.1	38.9 ± 0.7	1.7 ± 0.0	2.4	5.4
YOLO26s	640	48.6	47.8	87.2 ± 0.9	2.5 ± 0.0	9.5	20.7
YOLO26m	640	53.1	52.5	220.0 ± 1.4	4.7 ± 0.1	20.4	68.2
YOLO26l	640	55.0	54.4	286.2 ± 2.0	6.2 ± 0.2	24.8	86.4
YOLO26x	640	57.5	56.9	525.8 ± 4.0	11.8 ± 0.2	55.7	193.9

Instance Segmentation							
Model	**Size (px)**	**mAP$_{box}$ 50–95 (e2e)**	**mAP$_{mask}$ 50–95 (e2e)**	**Speed CPU ONNX (ms)**	**Speed T4 TRT10 (ms)**	**Params (M)**	**FLOPs (B)**
YOLO26n-seg	640	39.6	33.9	53.3 ± 0.5	2.1 ± 0.0	2.7	9.1
YOLO26s-seg	640	47.3	40.0	118.4 ± 0.9	3.3 ± 0.0	10.4	34.2
YOLO26m-seg	640	52.5	44.1	328.2 ± 2.4	6.7 ± 0.1	23.6	121.5
YOLO26l-seg	640	54.4	45.5	387.0 ± 3.7	8.0 ± 0.1	28.0	139.8
YOLO26x-seg	640	56.5	47.0	787.0 ± 6.8	16.4 ± 0.1	62.8	313.5

3.2 Classification performance metrics (ImageNet)

Table 3 summarizes the ImageNet classification performance of YOLO26 across model scales. As model capacity increases from YOLO26n to YOLO26x, Top-1 accuracy improves steadily from 71.4% to 79.9%, while maintaining strong Top-5 accuracy above 90% for all variants. Importantly, this accuracy scaling is achieved with predictable latency growth, as TensorRT FP16 inference remains below 4 ms even for the largest model. The compact FLOPs and parameter counts reported in Table 3 highlight that YOLO26 classification heads preserve efficiency, making them well suited for real-time image recognition on edge and embedded platforms.

3.3 Pose Performance Metrics (COCO)

Table 4 presents the pose estimation performance of YOLO26 on the COCO dataset. Across model scales, YOLO26 exhibits consistent gains in mAP$_{pose}$, increasing from 57.2% for the nano variant to 71.6% for the extra-large model under end-to-end evaluation. This accuracy improvement is accompanied by predictable scaling in latency and

Table 3: YOLO26 image classification performance on ImageNet at 224 px resolution. Results report Top-1/Top-5 accuracy, inference speed on CPU (ONNX) and NVIDIA T4 (TensorRT10 FP16), along with model size and FLOPs.

Model	Size (px)	Acc Top-1	Acc Top-5	Speed CPU ONNX (ms)	Speed T4 TRT10 (ms)	Params (M)	FLOPs (B @224)
YOLO26n-cls	224	71.4	90.1	5.0 ± 0.3	1.1 ± 0.0	2.8	0.5
YOLO26s-cls	224	76.0	92.9	7.9 ± 0.2	1.3 ± 0.0	6.7	1.6
YOLO26m-cls	224	78.1	94.2	17.2 ± 0.4	2.0 ± 0.0	11.6	4.9
YOLO26l-cls	224	79.0	94.6	23.2 ± 0.3	2.8 ± 0.0	14.1	6.2
YOLO26x-cls	224	79.9	95.0	41.4 ± 0.9	3.8 ± 0.0	29.6	13.6

computational cost, while maintaining real-time inference on GPU and near-real-time performance on CPU. The results in Table 4 demonstrate that YOLO26 effectively extends its efficiency-oriented design to pose estimation, making it suitable for real-time human and object keypoint analysis on both edge and server platforms.

Table 4: YOLO26 pose estimation performance on the COCO dataset at 640 px resolution. Results report end-to-end (e2e) pose accuracy, inference speed on CPU (ONNX) and NVIDIA T4 (TensorRT10 FP16), along with model size and FLOPs.

Model	Size (px)	mAP$_{pose}$ 50–95 (e2e)	mAP$_{pose}$ 50 (e2e)	Speed CPU ONNX (ms)	Speed T4 TRT10 (ms)	Params (M)	FLOPs (B)
YOLO26n-pose	640	57.2	83.3	40.3 ± 0.5	1.8 ± 0.0	2.9	7.5
YOLO26s-pose	640	63.0	86.6	85.3 ± 0.9	2.7 ± 0.0	10.4	23.9
YOLO26m-pose	640	68.8	89.6	218.0 ± 1.5	5.0 ± 0.1	21.5	73.1
YOLO26l-pose	640	70.4	90.5	275.4 ± 2.4	6.5 ± 0.1	25.9	91.3
YOLO26x-pose	640	71.6	91.6	565.4 ± 3.0	12.2 ± 0.2	57.6	201.7

3.4 Oriented Object Detection (OBB) Performance on DOTA v1

Table 5 reports the oriented object detection performance of YOLO26 on the DOTA v1 dataset. YOLO26 achieves consistent improvements in mAP$_{test}$ as model scale increases, reaching 56.7% mAP$_{50\text{–}95}$ for the extra-large variant under end-to-end evaluation. Despite the higher input resolution and computational demands of OBB tasks, YOLO26 maintains efficient inference, with sub-5 ms latency on GPU for small and medium models. The results in Table 5 demonstrate that YOLO26 effectively extends its edge-optimized, NMS-free design to rotated object detection, making it well suited for aerial imagery and remote sensing applications.

Table 5: YOLO26 oriented object detection (OBB) performance on the DOTA v1 dataset at 1024 px resolution. Results report end-to-end (e2e) test accuracy, inference speed on CPU (ONNX) and NVIDIA T4 (TensorRT10 FP16), along with model size and FLOPs.

Model	Size (px)	mAP$_{test}$ 50–95 (e2e)	mAP$_{test}$ 50 (e2e)	Speed CPU ONNX (ms)	Speed T4 TRT10 (ms)	Params (M)	FLOPs (B)
YOLO26n-obb	1024	52.4	78.9	97.7 ± 0.9	2.8 ± 0.0	2.5	14.0
YOLO26s-obb	1024	54.8	80.9	218.0 ± 1.4	4.9 ± 0.1	9.8	55.1
YOLO26m-obb	1024	55.3	81.0	579.2 ± 3.8	10.2 ± 0.3	21.2	183.3
YOLO26l-obb	1024	56.2	81.6	735.6 ± 3.1	13.0 ± 0.2	25.6	230.0
YOLO26x-obb	1024	56.7	81.7	1485.7 ± 11.5	30.5 ± 0.9	57.6	516.5

4 Real-Time Deployment with Ultralytics YOLO26

Over the past decade, the evolution of object detection models has been marked not only by increases in accuracy but also by growing complexity in deployment [26, 27, 28]. Early detectors such as R-CNN and its faster variants (Fast R-CNN, Faster R-CNN) achieved impressive detection quality but were computationally expensive, requiring multiple stages for region proposal and classification [29, 30, 31]. This limited their use in real-time and embedded applications. The arrival of the YOLO family transformed this landscape by reframing detection as a single regression problem, enabling real-time performance on commodity GPUs [32]. However, as the YOLO lineage progressed from YOLOv1 through YOLOv13, accuracy improvements often came at the cost of additional architectural components such as Distribution Focal Loss (DFL), complex post-processing steps like Non-Maximum Suppression (NMS), and

increasingly heavy backbones that introduced friction during deployment. YOLO26 addresses this longstanding challenge directly by streamlining both architecture and export pathways, thereby reducing deployment barriers across diverse hardware and software ecosystems.

4.1 Flexible Export and Integration Pathways

A key advantage of YOLO26 is its seamless integration into existing production pipelines. Ultralytics maintains an actively developed Python package that provides unified support for training, validation, and export, lowering the technical barrier for practitioners seeking to adopt YOLO26. Unlike earlier YOLO models, which required extensive custom conversion scripts for hardware acceleration [33, 34, 35], YOLO26 natively supports a wide range of export formats. These include TensorRT for maximum GPU acceleration, ONNX for broad cross-platform compatibility, CoreML for native iOS integration, TFLite for Android and edge devices, and OpenVINO for optimized performance on Intel hardware. The breadth of these export options enables researchers, engineers, and developers to move models from prototyping to production without encountering the compatibility bottlenecks common in earlier generations.

Historically, YOLOv3 through YOLOv7 often required manual intervention during export, particularly when targeting specialized inference engines such as NVIDIA TensorRT or Apple CoreML [36, 37]. Similarly, transformer-based detectors like DETR and its successors faced challenges when converted outside PyTorch environments due to their reliance on dynamic attention mechanisms. By comparison, YOLO26's architecture, simplified through the removal of DFL and the adoption of an NMS-free prediction head, ensures compatibility across platforms without sacrificing accuracy. This makes YOLO26 one of the most deployment-friendly detectors released to date, reinforcing its identity as an edge-first model.

4.2 Quantization and Resource-Constrained Devices

Beyond export flexibility, the true challenge in real-world deployment lies in ensuring efficiency on devices with limited computational resources [27, 38]. Edge devices such as smartphones, drones, and embedded vision systems often lack discrete GPUs and must balance memory, power, and latency constraints [39, 40]. Quantization is a widely adopted strategy to reduce model size and computational load, yet many complex detectors experience significant accuracy degradation under aggressive quantization. YOLO26 has been designed with this limitation in mind.

Owing to its streamlined architecture and simplified bounding box regression pipeline, YOLO26 demonstrates consistent accuracy under both half-precision (FP16) and integer (INT8) quantization schemes. FP16 quantization leverages native GPU support for mixed-precision arithmetic, enabling faster inference with reduced memory footprint. INT8 quantization compresses model weights to 8-bit integers, delivering dramatic reductions in model size and energy consumption while maintaining competitive accuracy. Benchmark experiments confirm that YOLO26 maintains stability across these quantization levels, outperforming YOLOv11 and YOLOv12 under identical conditions. This makes YOLO26 particularly well-suited for deployment on compact hardware such as NVIDIA Jetson Orin, Qualcomm Snapdragon AI accelerators, or even ARM-based CPUs powering smart cameras.

In contrast, transformer-based detectors such as RT-DETRv3 exhibit sharp drops in performance under INT8 quantization [41], primarily due to the sensitivity of attention mechanisms to reduced precision. Similarly, YOLOv12 and YOLOv13, while delivering strong accuracy on GPU servers, struggle to retain competitive performance on low-power devices once quantized. YOLO26 therefore establishes a new benchmark for quantization-aware design in object detection, demonstrating that architectural simplicity can directly translate into deployment robustness.

4.3 Cross-Industry Applications: From Robotics to Manufacturing

The practical impact of these deployment enhancements is best illustrated through cross-industry applications. In robotics, real-time perception is crucial for navigation, manipulation, and safe human-robot collaboration [42, 43]. By offering NMS-free predictions and consistent low-latency inference, YOLO26 allows robotic systems to interpret their environments faster and more reliably. For example, robotic arms equipped with YOLO26 can identify and grasp objects with higher precision under dynamic conditions, while mobile robots benefit from improved obstacle recognition in cluttered spaces. Compared with YOLOv8 or YOLOv11, YOLO26 offers reduced inference delay, which can be the difference between a safe maneuver and a collision in high-speed scenarios.

In manufacturing, YOLO26 has significant implications for automated defect detection and quality assurance. Traditional manual inspection is not only labor-intensive but also prone to human error. Previous YOLO releases, particularly YOLOv8, were already deployed in smart factories; however, the complexity of export and the latency overhead of NMS sometimes constrained large-scale rollout. YOLO26 mitigates these barriers by offering lightweight deployment options through OpenVINO or TensorRT, allowing manufacturers to integrate real-time defect detection systems directly on

production lines. Early benchmarks suggest that YOLO26-based defect detection pipelines achieve higher throughput and lower operational costs compared to both YOLOv12 and transformer-based alternatives such as DEIM.

4.4 Broader Insights from YOLO26 Deployment

Taken together, the deployment features of YOLO26 underscore a central theme in the evolution of object detection: architectural efficiency is just as critical as accuracy. While the past five years have seen the rise of increasingly sophisticated models ranging from convolution-based YOLO variants to transformer-based detectors like DETR and RT-DETR the gap between laboratory performance and production readiness has often limited their impact. YOLO26 bridges this gap by simplifying architecture, expanding export compatibility, and ensuring resilience under quantization, thereby aligning cutting-edge accuracy with practical deployment needs.

For developers building mobile applications, YOLO26 enables seamless integration through CoreML and TFLite, ensuring that models run natively on iOS and Android platforms. For enterprises deploying vision AI in cloud or on-premise servers, TensorRT and ONNX exports provide scalable acceleration options. For industrial and edge users, OpenVINO and INT8 quantization guarantee that performance remains consistent even under tight resource constraints. In this sense, YOLO26 is not only a step forward in object detection research but also a major milestone in democratizing deployment.

5 Conclusion and Future Directions

In conclusion, YOLO26 represents a significant leap in the YOLO object detection series, blending architectural innovation with a pragmatic focus on deployment. The model simplifies its design by removing the Distribution Focal Loss (DFL) module and eliminating the need for non-maximum suppression. By removing DFL, YOLO26 streamlines bounding box regression and avoids export complications, which broadens compatibility with various hardware. Likewise, its end-to-end, NMS-free inference enables the network to output final detections directly without a post-processing step. This not only reduces latency but also simplifies the deployment pipeline, making YOLO26 a natural evolution of earlier YOLO concepts. In training, YOLO26 introduces Progressive Loss Balancing (ProgLoss) and Small-Target-Aware Label Assignment (STAL), which together stabilize learning and boost accuracy on challenging small objects. Additionally, a novel MuSGD optimizer, combining properties of SGD and Muon, accelerates convergence and improves training stability. These enhancements work in concert to deliver a detector that is not only more accurate and robust but also markedly faster and lighter in practice.

Benchmark comparisons underscore YOLO26's strong performance relative to both its YOLO predecessors and contemporary models. Prior YOLO versions such as YOLO11 surpassed earlier releases with greater efficiency, and YOLO12 extended accuracy further through the integration of attention mechanisms. YOLO13 added hypergraph-based refinements to achieve additional improvements. Against transformer-based rivals, YOLO26 closes much of the gap. Its native NMS-free design mirrors the end-to-end approach of transformer-inspired detectors, but with YOLO's hallmark efficiency. YOLO26 delivers competitive accuracy while dramatically boosting throughput on common hardware and minimizing complexity. In fact, YOLO26's design yields up to 43% faster inference on CPU than previous YOLO versions, making it one of the most practical real-time detectors for resource-constrained environments. This harmonious balance of performance and efficiency allows YOLO26 to excel not just on benchmark leaderboards but also in actual field deployments where speed, memory, and energy are at a premium.

A major contribution of YOLO26 is its emphasis on deployment advantages. The model's architecture was deliberately optimized for real-world use: by omitting DFL and NMS, YOLO26 avoids operations that are difficult to implement on specialized hardware accelerators, thereby improving compatibility across devices. The network is exportable to a wide array of formats including ONNX, TensorRT, CoreML, TFLite, and OpenVINO ensuring that developers can integrate it into mobile apps, embedded systems, or cloud services with equal ease. Crucially, YOLO26 also supports robust quantization: it can be deployed with INT8 quantization or half-precision FP16 with minimal impact on accuracy, thanks to its simplified architecture that tolerates low-bitwidth inference. This means models can be compressed and accelerated while still delivering reliable detection performance. Such features translate to real edge performance gains from drones to smart cameras, YOLO26 can run real-time on CPU and small devices where previous YOLO models struggled. All these improvements demonstrate an overarching theme: YOLO26 bridges the gap between cutting-edge research ideas and deployable AI solutions. This approach underscores YOLO26's role as a bridge between academic innovation and industry application, bringing the latest vision advancements directly into the hands of practitioners.

5.1 Future Directions

Looking ahead, the trajectory of YOLO and object detection research suggests several promising directions. One clear avenue is the unification of multiple vision tasks into even more holistic models. YOLO26 already supports object detection, instance segmentation, pose estimation, oriented bounding boxes, and classification in one framework, reflecting a trend toward multi-task versatility. Future YOLO iterations might push this further by incorporating open-vocabulary and foundation-model capabilities. This could mean leveraging powerful vision-language models so that detectors can recognize arbitrary object categories in a zero-shot manner, without being limited to a fixed label set. By building on foundation models and large-scale pretraining, the next generation of YOLO could serve as a general-purpose vision AI that seamlessly handles detection, segmentation, and even description of novel objects in context.

Another key evolution is likely in the realm of semi-supervised and self-supervised learning for object detection [44, 45, 46, 47]. State-of-the-art detectors still rely heavily on large labeled datasets, but research is rapidly advancing methods to train on unlabeled or partially labeled data. Techniques such as teacher–student training [48, 49, 50], pseudo-labeling [51, 52], and self-supervised feature learning [53]could be integrated into the YOLO training pipeline to reduce the need for extensive manual annotations. A future YOLO might automatically leverage vast amounts of unannotated images or videos to improve recognition robustness. By doing so, the model can continue to improve its detection capabilities without proportional increases in labeled data, making it more adaptable to new domains or rare object categories.

Architecturally, we anticipate a continued blending of transformer and CNN design principles in object detectors. The success of recent YOLO models has shown that injecting attention and global reasoning into YOLO-like architectures can yield accuracy gains [54, 55]. Future YOLO architectures may adopt hybrid designs that combine convolutional backbones (for efficient local feature extraction) with transformer-based modules or decoders (for capturing long-range dependencies and context). Such hybrid approaches can improve how the model understands complex scenes, for example in crowded or highly contextual environments, by modeling relationships that pure CNNs or naive self-attention might miss. We expect next-generation detectors to intelligently fuse these techniques, achieving both rich feature representation and low latency. In short, the line between "CNN-based" and "transformer-based" detectors will continue to blur, taking the best of both worlds to handle diverse detection challenges.

Finally, as deployment becomes increasingly critical, future research is expected to emphasize edge-aware training and optimization from the outset. Rather than treating hardware constraints as a post hoc consideration, model design will increasingly co-evolve with target platforms through techniques such as quantization-aware training, automated model compression, and hardware-guided architecture search. Incorporating deployment feedback, including latency and energy measurements, directly into the training loop may further improve real-world efficiency. Such approaches could enable YOLO models to adapt their depth, resolution, or precision dynamically under runtime constraints, or be distilled into compact variants with minimal accuracy loss. This edge-first design philosophy is essential for sustaining real-time performance across IoT, AR/VR, and autonomous systems operating under strict resource limitations.

References

[1] Zhong-Qiu Zhao, Peng Zheng, Shou-tao Xu, and Xindong Wu. Object detection with deep learning: A review. *IEEE transactions on neural networks and learning systems*, 30(11):3212–3232, 2019.

[2] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years: A survey. *Proceedings of the IEEE*, 111(3):257–276, 2023.

[3] Chhavi Rana et al. Artificial intelligence based object detection and traffic prediction by autonomous vehicles–a review. *Expert Systems with Applications*, 255:124664, 2024.

[4] Zohaib Khan, Yue Shen, and Hui Liu. Objectdetection in agriculture: A comprehensive review of methods, applications, challenges, and future directions. *Agriculture*, 15(13):1351, 2025.

[5] Ranjan Sapkota, Marco Flores-Calero, Rizwan Qureshi, Chetan Badgujar, Upesh Nepal, Alwin Poulose, Peter Zeno, Uday Bhanu Prakash Vaddevolu, Sheheryar Khan, Maged Shoman, et al. Yolo advances to its genesis: a decadal and comprehensive review of the you only look once (yolo) series. *Artificial Intelligence Review*, 58(9):274, 2025.

[6] Ranjan Sapkota, Rahul Harsha Cheppally, Ajay Sharda, and Manoj Karkee. Rf-detr object detection vs yolov12: A study of transformer-based and cnn-based architectures for single-class and multi-class greenfruit detection in complex orchard environments under label ambiguity. *arXiv preprint arXiv:2504.13099*, 2025.

[7] Ranjan Sapkota, Dawood Ahmed, and Manoj Karkee. Comparing yolov8 and mask r-cnn for instance segmentation in complex orchard environments. *Artificial Intelligence in Agriculture*, 13:84–99, 2024.

[8] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.

[9] Joseph Redmon and Ali Farhadi. Yolo9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7263–7271, 2017.

[10] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[11] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[12] Chuyi Li, Lulu Li, Hongliang Jiang, Kaiheng Weng, Yifei Geng, Liang Li, Zaidan Ke, Qingyuan Li, Meng Cheng, Weiqiang Nie, et al. Yolov6: A single-stage object detection framework for industrial applications. *arXiv preprint arXiv:2209.02976*, 2022.

[13] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7464–7475, 2023.

[14] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. In *European conference on computer vision*, pages 1–21. Springer, 2024.

[15] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, et al. Yolov10: Real-time end-to-end object detection. *Advances in Neural Information Processing Systems*, 37:107984–108011, 2024.

[16] Yunjie Tian, Qixiang Ye, and David Doermann. Yolov12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524*, 2025.

[17] Mengqi Lei, Siqi Li, Yihong Wu, Han Hu, You Zhou, Xinhu Zheng, Guiguang Ding, Shaoyi Du, Zongze Wu, and Yue Gao. Yolov13: Real-time object detection with hypergraph-enhanced adaptive visual perception. *arXiv preprint arXiv:2506.17733*, 2025.

[18] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.

[19] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *IEEE transactions on pattern analysis and machine intelligence*, 39(6):1137–1149, 2016.

[20] Tausif Diwan, G Anirudh, and Jitendra V Tembhurne. Object detection using yolo: challenges, architectural successors, datasets and applications. *multimedia Tools and Applications*, 82(6):9243–9275, 2023.

[21] Momina Liaqat Ali and Zhou Zhang. The yolo framework: A comprehensive review of evolution, applications, and benchmarks in object detection. *Computers*, 13(12):336, 2024.

[22] Kyriakos D Apostolidis and George A Papakostas. Delving into yolo object detection models: Insights into adversarial robustness. *Electronics*, 14(8):1624, 2025.

[23] Enerst Edozie, Aliyu Nuhu Shuaibu, Ukagwu Kelechi John, and Bashir Olaniyi Sadiq. Comprehensive review of recent developments in visual object detection based on deep learning. *Artificial Intelligence Review*, 58(9):277, 2025.

[24] Mupparaju Sohan, Thotakura Sai Ram, and Ch Venkata Rami Reddy. A review on yolov8 and its advancements. In *International Conference on Data Intelligence and Cognitive Informatics*, pages 529–545. Springer, 2024.

[25] Javaria Farooq, Muhammad Muaz, Khurram Khan Jadoon, Nayyer Aafaq, and Muhammad Khizer Ali Khan. An improved yolov8 for foreign object debris detection with optimized architecture for small objects. *Multimedia Tools and Applications*, 83(21):60921–60947, 2024.

[26] Maria Trigka and Elias Dritsas. A comprehensive survey of machine learning techniques and models for object detection. *Sensors*, 25(1):214, 2025.

[27] Md Tanzib Hosain, Asif Zaman, Mushfiqur Rahman Abir, Shanjida Akter, Sawon Mursalin, and Shadman Sakeeb Khan. Synchronizing object detection: Applications, advancements and existing challenges. *IEEE access*, 12:54129–54167, 2024.

[28] Ambati Pravallika, Mohammad Farukh Hashmi, and Aditya Gupta. Deep learning frontiers in 3d object detection: a comprehensive review for autonomous driving. *IEEE Access*, 2024.

[29] Jiawei Tian, Seungho Lee, and Kyungtae Kang. Faster r-cnn in healthcare and disease detection: A comprehensive review. In *2025 International Conference on Electronics, Information, and Communication (ICEIC)*, pages 1–6. IEEE, 2025.

[30] Peng Fu and Jiyang Wang. Lithology identification based on improved faster r-cnn. *Minerals*, 14(9):954, 2024.

[31] Samiyaa Yaseen Mohammed. Architecture review: Two-stage and one-stage object detection. *Franklin Open*, page 100322, 2025.

[32] Richard Johnson. *YOLO Object Detection Explained: Definitive Reference for Developers and Engineers*. HiTeX Press, 2025.

[33] Daniel Pestana, Pedro R Miranda, João D Lopes, Rui P Duarte, Mário P Véstias, Horácio C Neto, and José T De Sousa. A full featured configurable accelerator for object detection with yolo. *IEEE Access*, 9:75864–75877, 2021.

[34] Duy Thanh Nguyen, Tuan Nghia Nguyen, Hyun Kim, and Hyuk-Jae Lee. A high-throughput and power-efficient fpga implementation of yolo cnn for object detection. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(8):1861–1873, 2019.

[35] Caiwen Ding, Shuo Wang, Ning Liu, Kaidi Xu, Yanzhi Wang, and Yun Liang. Req-yolo: A resource-aware, efficient quantization framework for object detection on fpgas. In *proceedings of the 2019 ACM/SIGDA international symposium on field-programmable gate arrays*, pages 33–42, 2019.

[36] Patricia Citranegara Kusuma and Benfano Soewito. Multi-object detection using yolov7 object detection algorithm on mobile device. *Journal of Applied Engineering and Technological Science (JAETS)*, 5(1):305–320, 2023.

[37] Nico Surantha and Nana Sutisna. Key considerations for real-time object recognition on edge computing devices. *Applied Sciences*, 15(13):7533, 2025.

[38] Kareemah Abdulhaq and Abdussalam Ali Ahmed. Real-time object detection and recognition in embedded systems using open-source computer vision frameworks. *Int. J. Electr. Eng. and Sustain.*, pages 103–118, 2025.

[39] Sabir Hossain and Deok-jin Lee. Deep learning-based real-time multiple-object detection and tracking from aerial imagery via a flying robot with gpu-based embedded devices. *Sensors*, 19(15):3371, 2019.

[40] Arief Setyanto, Theopilus Bayu Sasongko, Muhammad Ainul Fikri, and In Kee Kim. Near-edge computing aware object detection: A review. *IEEE Access*, 12:2989–3011, 2023.

[41] Shuo Wang, Chunlong Xia, Feng Lv, and Yifeng Shi. Rt-detrv3: Real-time end-to-end object detection with hierarchical dense positive supervision. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1628–1636. IEEE, 2025.

[42] Andrea Bonci, Pangcheng David Cen Cheng, Marina Indri, Giacomo Nabissi, and Fiorella Sibona. Human-robot perception in industrial environments: A survey. *Sensors*, 21(5):1571, 2021.

[43] Ranjan Sapkota and Manoj Karkee. Object detection with multimodal large vision-language models: An in-depth review. *Information Fusion*, 126:103575, 2026.

[44] Peng Tang, Chetan Ramaiah, Yan Wang, Ran Xu, and Caiming Xiong. Proposal learning for semi-supervised object detection. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2291–2301, 2021.

[45] Kihyuk Sohn, Zizhao Zhang, Chun-Liang Li, Han Zhang, Chen-Yu Lee, and Tomas Pfister. A simple semi-supervised learning framework for object detection. *arXiv preprint arXiv:2005.04757*, 2020.

[46] Gabriel Huang, Issam Laradji, David Vazquez, Simon Lacoste-Julien, and Pau Rodriguez. A survey of self-supervised and few-shot object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(4):4071–4089, 2022.

[47] Veenu Rani, Syed Tufael Nabi, Munish Kumar, Ajay Mittal, and Krishan Kumar. Self-supervised learning: A succinct review. *Archives of Computational Methods in Engineering*, 30(4):2761–2775, 2023.

[48] Yu-Jhe Li, Xiaoliang Dai, Chih-Yao Ma, Yen-Cheng Liu, Kan Chen, Bichen Wu, Zijian He, Kris Kitani, and Peter Vajda. Cross-domain adaptive teacher for object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7581–7590, 2022.

[49] Mengde Xu, Zheng Zhang, Han Hu, Jianfeng Wang, Lijuan Wang, Fangyun Wei, Xiang Bai, and Zicheng Liu. End-to-end semi-supervised object detection with soft teacher. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 3060–3069, 2021.

[50] Peng Mi, Jianghang Lin, Yiyi Zhou, Yunhang Shen, Gen Luo, Xiaoshuai Sun, Liujuan Cao, Rongrong Fu, Qiang Xu, and Rongrong Ji. Active teacher for semi-supervised object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14482–14491, 2022.

[51] Gang Li, Xiang Li, Yujie Wang, Yichao Wu, Ding Liang, and Shanshan Zhang. Pseco: Pseudo labeling and consistency training for semi-supervised object detection. In *European Conference on Computer Vision*, pages 457–472. Springer, 2022.

[52] Benjamin Caine, Rebecca Roelofs, Vijay Vasudevan, Jiquan Ngiam, Yuning Chai, Zhifeng Chen, and Jonathon Shlens. Pseudo-labeling for scalable 3d object detection. *arXiv preprint arXiv:2103.02093*, 2021.

[53] Longlong Jing and Yingli Tian. Self-supervised visual feature learning with deep neural networks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 43(11):4037–4058, 2020.

[54] Ming Kang, Chee-Ming Ting, Fung Fung Ting, and Raphael C-W Phan. Asf-yolo: A novel yolo model with attentional scale sequence fusion for cell instance segmentation. *Image and Vision Computing*, 147:105057, 2024.

[55] Ajantha Vijayakumar and Subramaniyaswamy Vairavasundaram. Yolo-based object detection models: A review and its applications. *Multimedia Tools and Applications*, 83(35):83535–83574, 2024.